Name : vaibhav jadhav

Batch:B4

class : syb-72

Aim: Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seab libraries ⌄ Scatter plot ⌄ Line plot ⌄ Bar plot ⌄ Histogram ⌄ Box plot ⌄ Pair plot

Q1. What is the importance of data visualization in data analysis? How does visualization help in exploratory data analysis (EDA)?

Answer: Data visualization is important in data analysis because it represents data in graphical fo making complex information easy to understand. It helps in quickly identifying patterns, trends, a relationships that are difficult to see in raw data.

How visualization helps in EDA:

Helps to identify patterns and trends Detects outliers and anomalies Shows relationships betwee variables Understands data distribution Makes analysis faster and easier

Q2. What is the difference between Matplotlib and other visualization libraries?

Answer: Matplotlib is a basic and widely used plotting library in Python that provides full control graphs, while other libraries like Seaborn and Plotly offer more advanced and visually attractive p simpler code.

Differences:

Matplotlib → low-level, more control over customization Seaborn → high-level, better design anc statistical plots Plotly → interactive and dynamic visualizations Matplotlib → requires more code libraries → easier and faster to use

Q3. What is the purpose of the pyplot module in Matplotlib?

Answer: The pyplot module in Matplotlib is used to create and manage plots easily using simple functions. It provides a MATLAB-like interface for drawing graphs and visualizing data.

Points:

Used to create different types of plots (line, bar, scatter, etc.) Manages figure and axes automatica Provides functions like plot(), show(), title() Makes plotting simple and quick Widely used for basi visualization

Q4. Explain the process of creating a basic line plot.

Answer: A basic line plot is used to show the relationship between two variables by connecting d points with a line. It is commonly used to display trends over time or continuous data.

Process (Steps):

Import matplotlib library (pyplot) Prepare or load the data (x and y values) Use plt.plot(x, y) to cre line plot Add labels using xlabel() and ylabel() Add title using title() Display the plot using show()

Q5. Why is customization important in data visualization? What customization options are availal Matplotlib?

Answer: Customization is important in data visualization because it makes graphs more clear, att and easy to understand. It helps highlight important information and improves the overall prese data.

Customization options in Matplotlib:

Titles (plt.title()) Axis labels (xlabel(), ylabel()) Colors and styles of lines Line width and line type (s dashed, etc.) Markers (circle, square, etc.) Legends (plt.legend()) Axis limits (xlim(), ylim()) Grid (plt

In [ ]:
```
from google.colab import files
uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Ple rerun this cell to enable.

```
Saving employee_performance_data.csv to employee_performance_data.csv
```

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```
df = pd.read_csv("employee_performance_data.csv")

# Show first 5 rows
df.head()
```

Out[ ]:

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	Experience	Perforn
0	101	Alice	50	Female	Marketing	2009-01-18	62606.0	13.0	
1	102	Bob	36	Female	Marketing	2012-01-01	41534.0	1.0	
2	103	Charlie	29	Female	IT	2007-02-15	NaN	9.0	
3	104	David	42	Male	Marketing	2015-12-29	31016.0	7.0	
4	105	Eve	40	Female	IT	2005-02-04	119789.0	9.0	

In [ ]:
```
df.info()
df.describe()
```

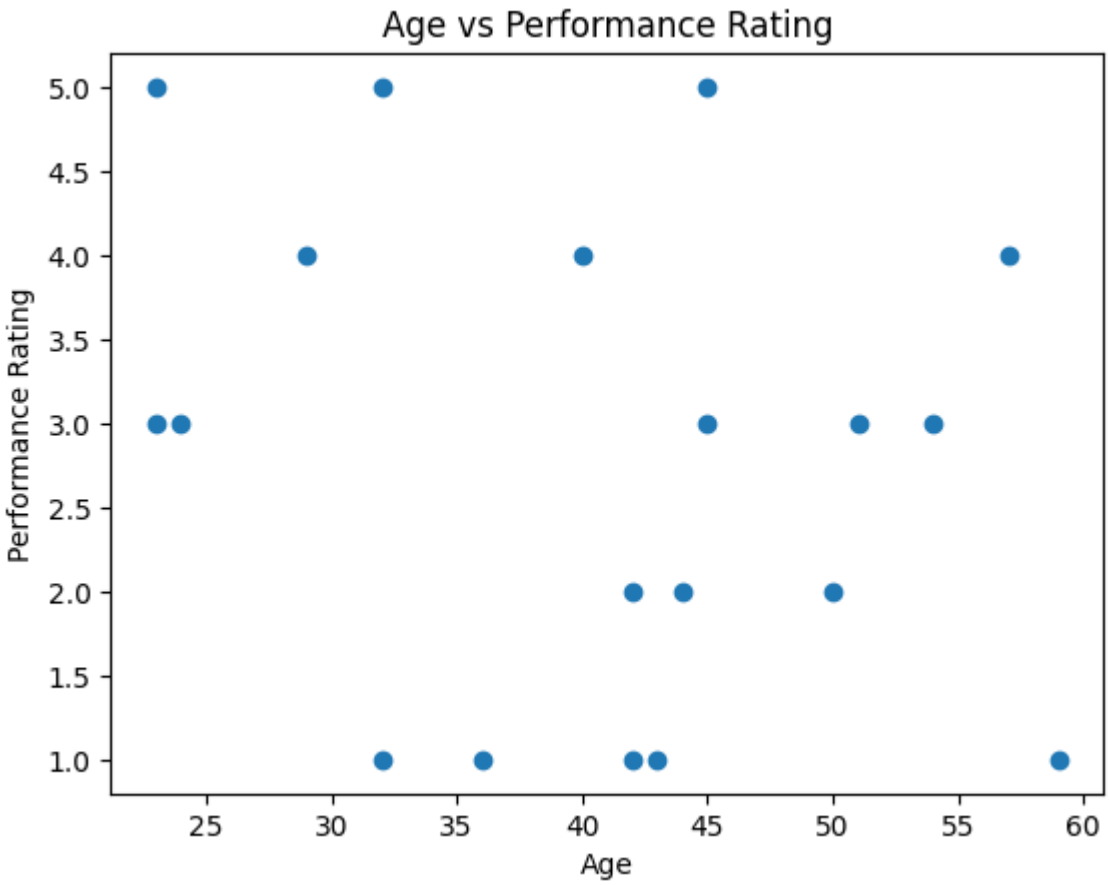
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Employee_ID          20 non-null     int64
 1   Name                 20 non-null     object
 2   Age                  20 non-null     int64
 3   Gender               20 non-null     object
 4   Department           20 non-null     object
 5   Join_Date            20 non-null     object
 6   Salary               19 non-null     float64
 7   Experience           19 non-null     float64
 8   Performance_Rating   19 non-null     float64
 9   Bonus                19 non-null     float64
 10  Work_Hours_Per_Week  19 non-null     float64
dtypes: float64(5), int64(2), object(4)
memory usage: 1.8+ KB
```

Out[ ]:

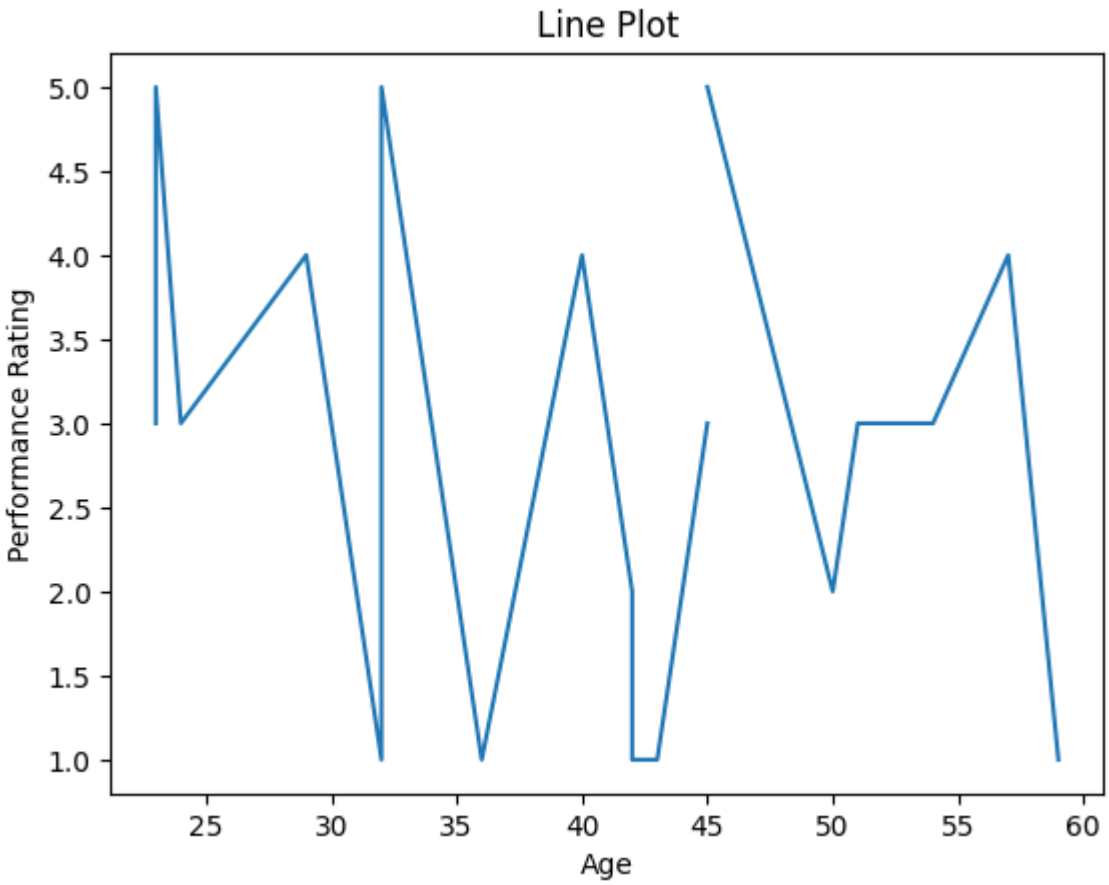
	Employee_ID	Age	Salary	Experience	Performance_Rating	Bonus
count	20.00000	20.000000	19.000000	19.000000	19.000000	19.000000
mean	110.50000	40.800000	76338.368421	6.526316	2.789474	11033.421053
std	5.91608	10.909195	31729.124187	4.114587	1.436777	3737.670818
min	101.00000	23.000000	31016.000000	1.000000	1.000000	5206.000000
25%	105.75000	32.000000	47716.000000	3.000000	1.500000	8639.000000
50%	110.50000	42.500000	69099.000000	8.000000	3.000000	11374.000000
75%	115.25000	46.250000	108431.500000	9.500000	4.000000	12647.500000
max	120.00000	59.000000	119812.000000	13.000000	5.000000	19820.000000

In [ ]:

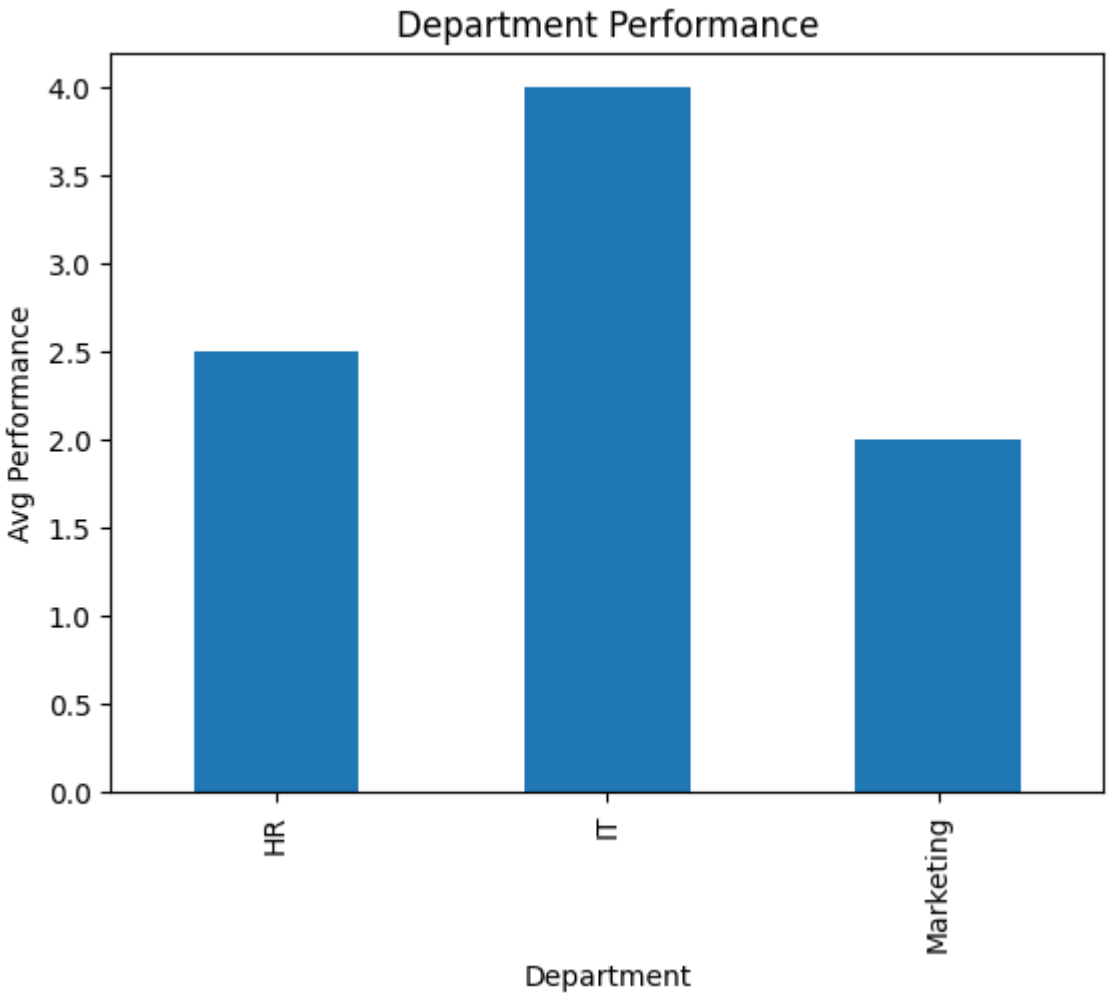
```
plt.scatter(df['Age'], df['Performance_Rating'])
plt.xlabel("Age")
plt.ylabel("Performance Rating")
plt.title("Age vs Performance Rating")
plt.show()
```



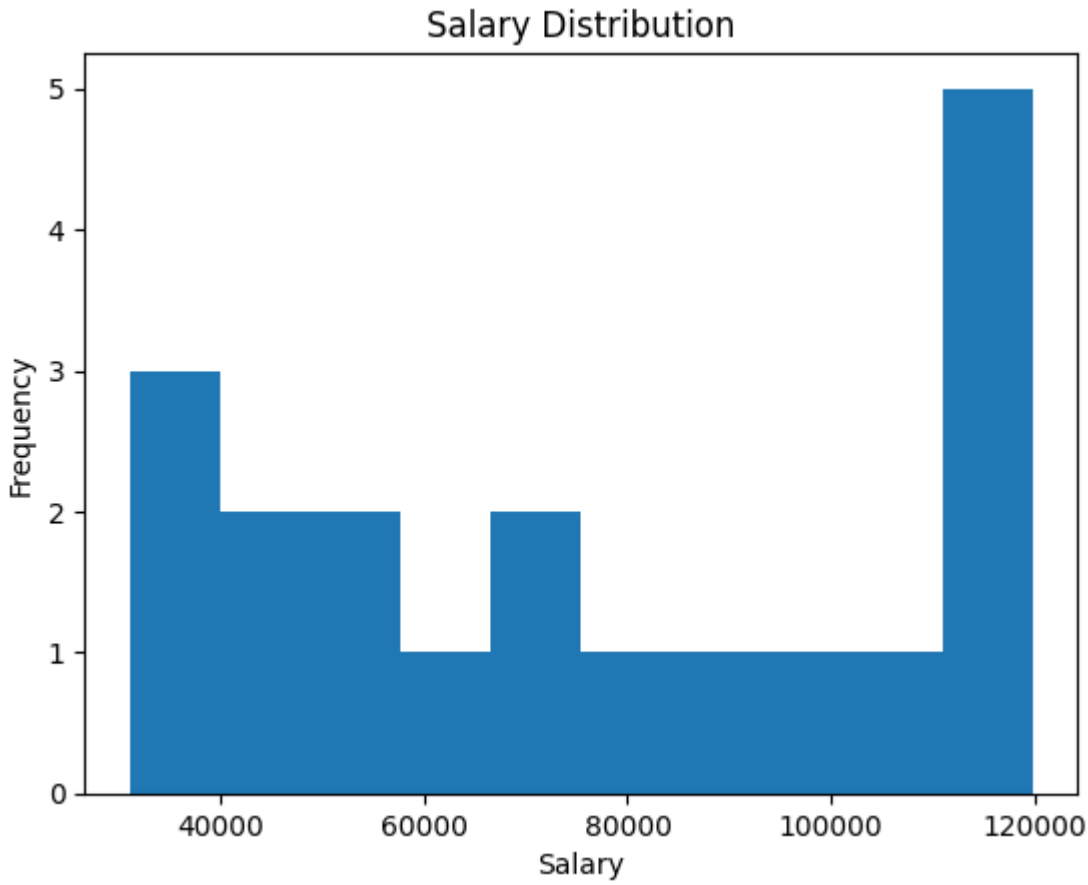
In [ ]:
```
df_sorted = df.sort_values(by='Age')

plt.plot(df_sorted['Age'], df_sorted['Performance_Rating'])
plt.xlabel("Age")
plt.ylabel("Performance Rating")
plt.title("Line Plot")
plt.show()
```



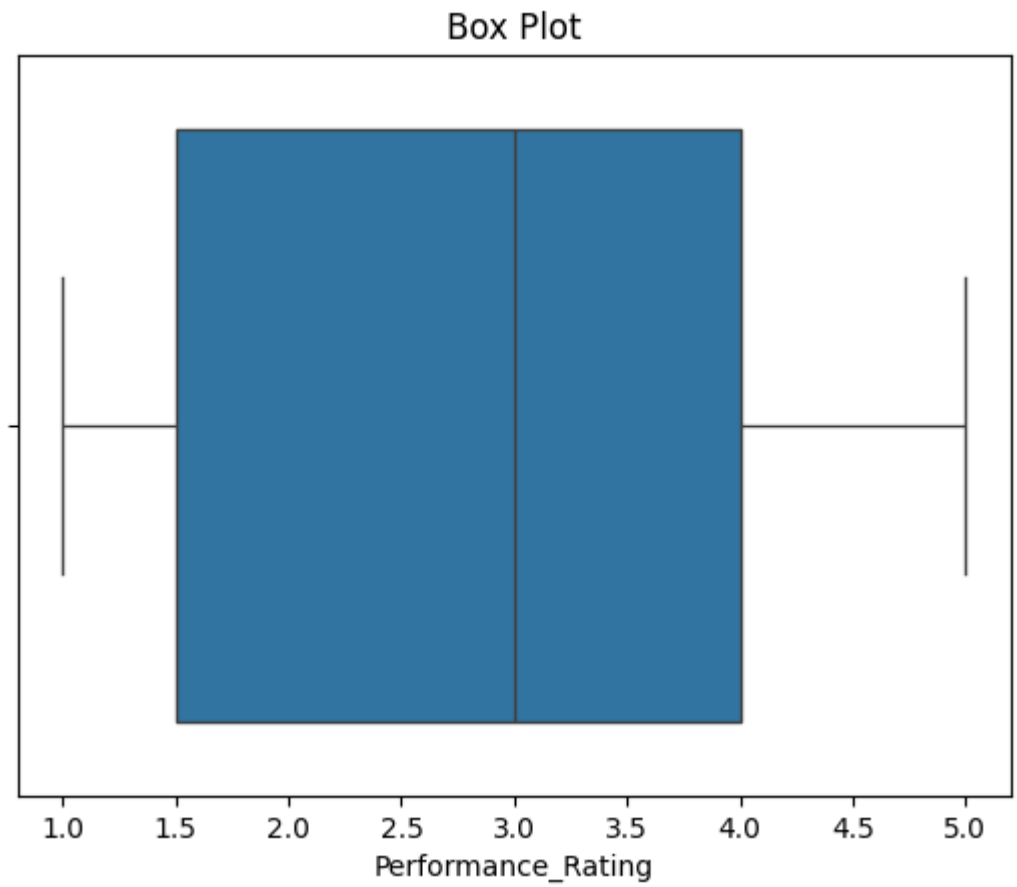
In [ ]:
```
dept_perf = df.groupby('Department')['Performance_Rating'].mean()

dept_perf.plot(kind='bar')
plt.xlabel("Department")
plt.ylabel("Avg Performance")
plt.title("Department Performance")
plt.show()
```

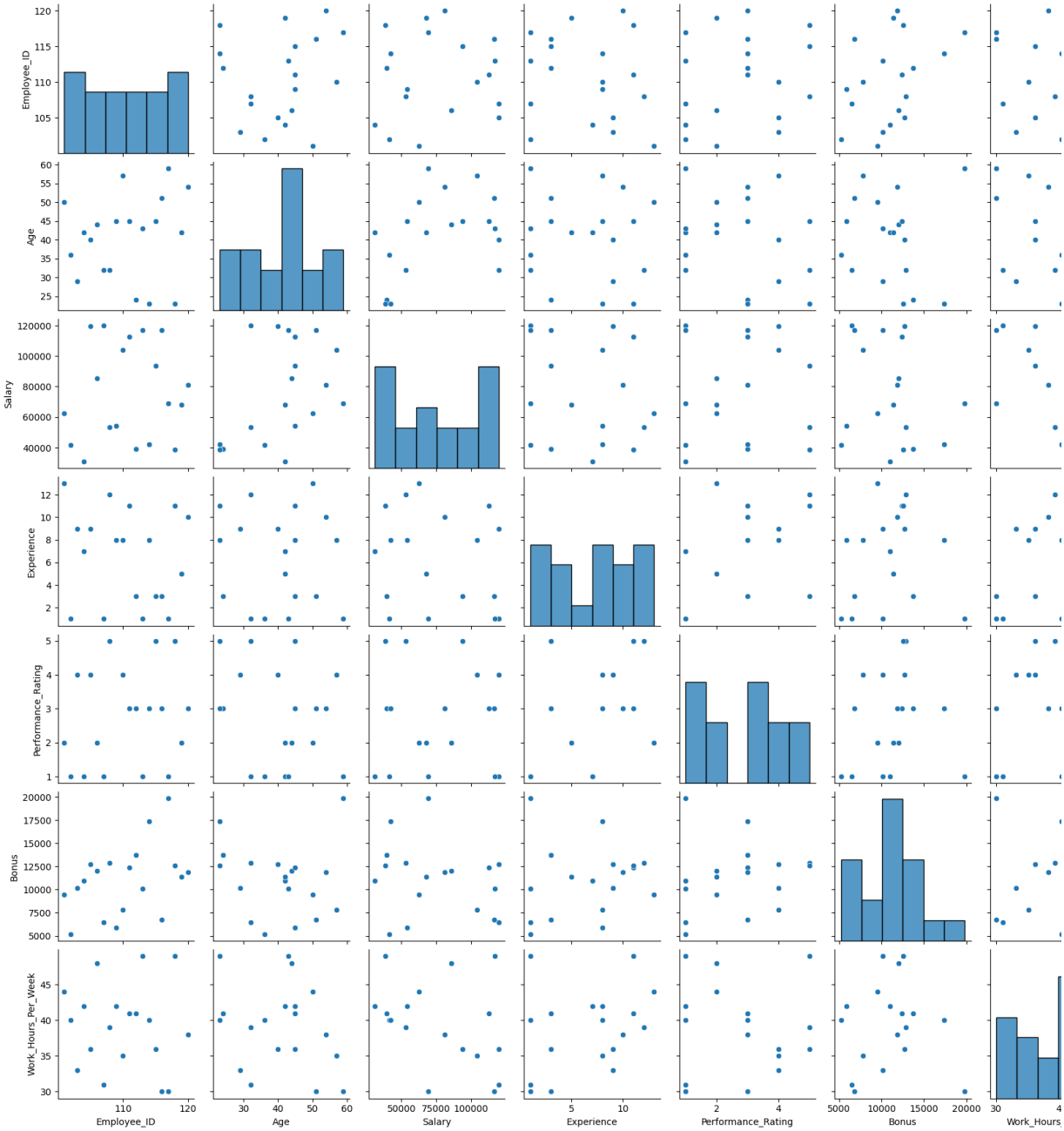


In [ ]:
```
plt.hist(df['Salary'], bins=10)
plt.xlabel("Salary")
plt.ylabel("Frequency")
plt.title("Salary Distribution")
plt.show()
```



In [ ]:
```
sns.boxplot(x=df['Performance_Rating'])
plt.title("Box Plot")
plt.show()
```



In [ ]:

```
sns.pairplot(df)
plt.show()
```